

09 - 多平台接入

阅读时间：约 2 小时
前置知识：08 - 技能与工具
学习目标：理解 Nanobot 的 MessageBus 架构、ChannelManager 策略、BaseChannel 适配器模式，掌握多平台接入方法

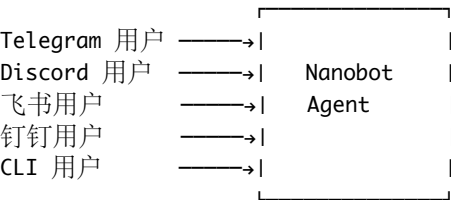
目录

- 9.1 多平台支持概述
- 9.2 MessageBus 架构详解
- 9.3 ChannelManager 出站策略
- 9.4 BaseChannel 适配器模式
- 9.5 各平台接入指南
- 9.6 session_key 会话隔离机制
- 9.7 多平台消息流完整链路
- 9.8 实战练习
- 9.9 面试高频题
- 9.10 本章小结

9.1 多平台支持概述

9.1.1 为什么需要多平台支持

在实际场景中，一个 AI Agent 需要同时服务于多个沟通渠道：



一个 Agent 实例可以同时接入多个平台，每个平台的用户独立会话互不干扰。

9.1.2 支持的平台列表

平台	类型	接入方式	适用场景
CLI	命令行	内置默认	开发调试
Telegram	IM	Bot Token	个人/小团队
Discord	IM	Bot Token	社区/游戏
飞书 (Feishu/Lark)	企业 IM	App ID/Secret	企业办公
钉钉 (DingTalk)	企业 IM	Robot 配置	企业办公
Slack	企业 IM	Bot Token	海外企业
微信 (WeChat)	IM	自定义接入	国内个人
QQ	IM	自定义接入	国内社交
Matrix	开源 IM	Access Token	自托管
Email	邮件	IMAP/SMTP	异步通知

9.1.3 核心理念

Nanobot 的多平台支持基于三层解耦架构：

- 层次 1: Agent 核心逻辑
 - └ 不关心消息来自哪个平台
 - └ 统一处理 InboundMessage
 - └ 统一生成 OutboundMessage

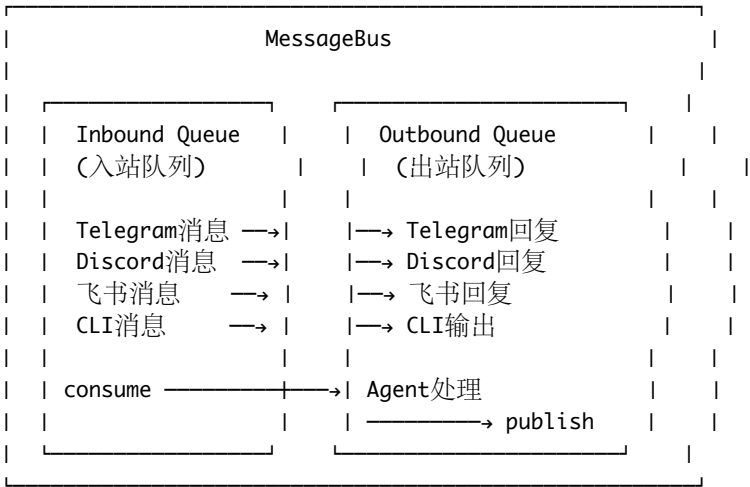
- 层次 2: MessageBus 消息总线
 - └ 异步双队列 (Inbound + Outbound)
 - └ 解耦生产者和消费者
 - └ 平台无关的消息格式

- 层次 3: Channel 适配器
 - └ 每个平台一个 Channel 实现
 - └ 负责协议转换
 - └ 将平台特定格式 → 统一格式

9.2 MessageBus 架构详解

9.2.1 双队列架构

MessageBus 是 Nanobot 多平台通信的核心枢纽，采用双队列设计：



9.2.2 InboundMessage 数据结构

InboundMessage 表示从任何平台进入的用户消息：

```

@dataclass
class InboundMessage:
    channel: str          # 来源通道标识，如 "telegram", "discord", "cli"
    sender_id: str        # 发送者 ID（平台内唯一标识）
    chat_id: str          # 会话 ID（群组 ID 或私聊 ID）
    content: str          # 文本内容
    media: list           # 媒体附件（图片、文件等）
    metadata: dict        # 平台特定的元数据
    session_key: str      # 会话键（用于会话隔离）
  
```

各字段详解：

字段	说明	示例
channel	消息来源平台	"telegram", "discord", "cli"
sender_id	发送者在该平台的唯一 ID	"user_123456"

字段	说明	示例
chat_id	会话标识（私聊或群组）	"chat_789", "group_456"
content	消息文本内容	" 你好，请帮我查天气"
media	附件列表	[{"type": "image", "url": "..."}]
metadata	平台特定数据	{"message_id": "123", "reply_to": "456"}
session_key	会话隔离键	"telegram:chat_789"

9.2.3 OutboundMessage 数据结构

OutboundMessage 表示 Agent 要发出的回复消息：

```
@dataclass
class OutboundMessage:
    channel: str        # 目标通道标识
    chat_id: str        # 目标会话 ID
    content: str        # 回复文本内容
    reply_to: str       # 回复的原消息 ID（可选）
    metadata: dict      # 平台特定的元数据
    media: list         # 媒体附件
```

9.2.4 MessageBus API

```
class MessageBus:
    """ 异步消息总线 """

    def __init__(self):
        self._inbound_queue = asyncio.Queue()
        self._outbound_queue = asyncio.Queue()

    # — 入站 API —

    async def publish_inbound(self, message: InboundMessage):
        """Channel 将用户消息发布到入站队列"""
        await self._inbound_queue.put(message)

    async def consume_inbound(self) -> InboundMessage:
        """Agent 从入站队列消费消息"""
        return await self._inbound_queue.get()

    # — 出站 API —

    async def publish_outbound(self, message: OutboundMessage):
        """Agent 将回复消息发布到出站队列"""
        await self._outbound_queue.put(message)

    async def consume_outbound(self) -> OutboundMessage:
        """Channel 从出站队列消费消息并发送"""
        return await self._outbound_queue.get()
```

9.2.5 为什么用队列而不是直接调用

对比	直接调用	队列解耦
耦合度	Agent 直接依赖 Channel	完全解耦
并发	需要手动管理	队列天然支持

对比	直接调用	队列解耦
背压	无	队列满时自动等待
扩展性	添加平台需要修改 Agent	只需添加 Channel
测试	需要 Mock 平台	直接操作队列

9.3 ChannelManager 出站策略

9.3.1 ChannelManager 的职责

ChannelManager 是 MessageBus 和各 Channel 之间的管理层，负责：

1. 管理所有已注册的 Channel
2. 路由出站消息到正确的 Channel
3. 处理流式输出的合并和优化
4. 处理发送失败的重试

Agent → MessageBus.outbound → ChannelManager → Channel.send()

|
├─ 流式合并
├─ 降频优化
├─ 失败重试
└─ 路由分发

9.3.2 _stream_delta 合并（降频优化）

LLM 的流式输出是逐 token 产生的，如果每个 token 都发送一条消息，会导致：

- Telegram/Discord 等平台的 API 限流
- 用户端消息闪烁
- 网络资源浪费

ChannelManager 通过 _stream_delta 合并机制优化：

```
class ChannelManager:
    async def _stream_delta(self, channel: str, chat_id: str,
                           content_delta: str):
        """ 合并流式输出片段，降低发送频率 """
        key = f"{channel}:{chat_id}"

        # 将新片段追加到缓冲区
        if key not in self._buffer:
            self._buffer[key] = ""
        self._buffer[key] += content_delta

        # 检查是否应该发送
        now = time.time()
        last_send = self._last_send_time.get(key, 0)

        # 降频策略：至少间隔 300ms 才发送一次
        if now - last_send >= 0.3:
            await self._flush_buffer(key)
            self._last_send_time[key] = now
```

LLM 输出流：

```
t0: "你"
t1: "好"
t2: "! "
```

t3: "我"
t4: "是"
t5: "一"
t6: "个"
t7: "AI"

无降频（8次发送）： "你" → "好" → "!" → "我" → "是" → "一" → "个" → "AI"

有降频（3次发送）： "你好! " → "我是一" → "个AI"
□ 300ms后 □ 300ms后 □ 结束flush

9.3.3 __progress / __tool_hint 过滤

当 Agent 正在执行工具调用或内部推理时，会产生进度信息和工具提示。ChannelManager 会根据配置决定是否将这些信息展示给用户：

```
async def _handle_progress(self, channel: str, chat_id: str,
                           progress_info: str):
    """ 处理进度信息 """
    channel_config = self._channels[channel]

    # 某些平台可能不展示进度（如 Email）
    if channel_config.get("show_progress", True):
        await self._send(channel, chat_id,
                        f"□ {progress_info}")

async def _handle_tool_hint(self, channel: str, chat_id: str,
                           tool_name: str):
    """ 处理工具调用提示 """
    channel_config = self._channels[channel]

    if channel_config.get("show_tool_hints", True):
        await self._send(channel, chat_id,
                        f"□ 正在使用 {tool_name}...")
```

9.3.4 发送失败指数退避

当向平台发送消息失败时（网络问题、API 限流等），ChannelManager 使用指数退避策略重试：

```
async def _send_with_retry(self, channel: str, chat_id: str,
                           content: str, max_retries: int = 5):
    """ 带指数退避的消息发送 """
    for attempt in range(max_retries):
        try:
            await self._channels[channel].send(chat_id, content)
            return # 发送成功
        except Exception as e:
            if attempt < max_retries - 1:
                # 指数退避: 1s, 2s, 4s, 8s, 16s
                wait_time = 2 ** attempt
                logger.warning(
                    f"Send failed (attempt {attempt + 1}), "
                    f"retrying in {wait_time}s: {e}")
            else:
                logger.error(f"Send failed after {max_retries} attempts: {e}")
            raise
```

重试时间线:

失败 → 等待 1s → 重试
失败 → 等待 2s → 重试
失败 → 等待 4s → 重试
失败 → 等待 8s → 重试
失败 → 等待 16s → 重试
失败 → 放弃, 记录错误日志

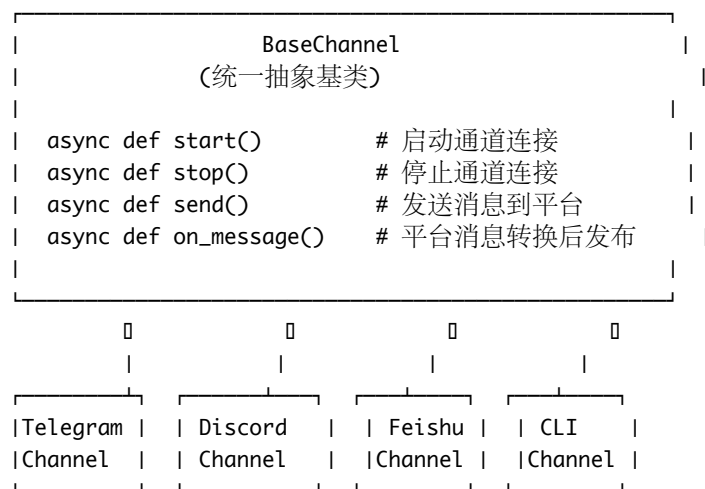
面试要点: 指数退避 (Exponential Backoff) 是分布式系统中处理瞬时故障的标准策略。配合最大重试次数和可选的抖动 (jitter), 可以有效避免“雷群效应” (Thundering Herd)。

9.4 BaseChannel 适配器模式

9.4.1 适配器模式 (Adapter Pattern)

每个平台的 API 都不一样 (Telegram 用 Bot API、Discord 用 Gateway、飞书用 Event API…), 但 Nanobot 的 Agent 核心不应该关心这些差异。

BaseChannel 通过适配器模式解决这个问题:



9.4.2 BaseChannel 抽象基类

```
from abc import ABC, abstractmethod
```

```
class BaseChannel(ABC):
    """ 所有平台通道的抽象基类 """

    def __init__(self, name: str, config: dict, message_bus: MessageBus):
        self.name = name
        self.config = config
        self.bus = message_bus

    @abstractmethod
    async def start(self):
        """ 启动通道 (建立连接、注册 Webhook 等) """
        pass

    @abstractmethod
    async def stop(self):
        """ 停止通道 (断开连接、清理资源) """
```

```

    pass

@abstractmethod
async def send(self, chat_id: str, content: str,
               media: list = None):
    """ 发送消息到该平台 """
    pass

async def on_message(self, platform_message: dict):
    """ 收到平台消息后，转换为 InboundMessage 并发布 """
    inbound = self._convert_to_inbound(platform_message)
    await self.bus.publish_inbound(inbound)

@abstractmethod
def _convert_to_inbound(self, platform_message: dict) -> InboundMessage:
    """ 将平台特定消息格式转换为统一的 InboundMessage """
    pass

```

9.4.3 Telegram Channel 实现示例

```

class TelegramChannel(BaseChannel):
    """Telegram 平台适配器"""

    def __init__(self, config: dict, message_bus: MessageBus):
        super().__init__("telegram", config, message_bus)
        self.bot_token = config["bot_token"]
        self.bot = None

    async def start(self):
        """ 启动 Telegram Bot """
        from telegram import Bot
        self.bot = Bot(token=self.bot_token)
        # 注册消息处理回调
        # 开始轮询或 Webhook

    async def stop(self):
        """ 停止 Telegram Bot """
        if self.bot:
            await self.bot.shutdown()

    async def send(self, chat_id: str, content: str,
                  media: list = None):
        """ 发送消息到 Telegram """
        if media:
            for item in media:
                if item["type"] == "image":
                    await self.bot.send_photo(
                        chat_id=chat_id,
                        photo=item["url"]
                    )

        if content:
            await self.bot.send_message(
                chat_id=chat_id,
                text=content,
                parse_mode="Markdown"
            )

```

```

def _convert_to_inbound(self, update: dict) -> InboundMessage:
    """ 将 Telegram Update 转换为 InboundMessage """
    message = update.get("message", {})
    return InboundMessage(
        channel="telegram",
        sender_id=str(message["from"]["id"]),
        chat_id=str(message["chat"]["id"]),
        content=message.get("text", ""),
        media=self._extract_media(message),
        metadata={
            "message_id": message["message_id"],
            "username": message["from"].get("username"),
        },
        session_key=f"telegram:{message['chat']['id']}"
    )

```

9.4.4 添加新平台的步骤

如果你要为 Nanobot 添加一个新平台（比如 Line），只需：

1. 创建 Channel 类，继承 BaseChannel

```

class LineChannel(BaseChannel):
    def __init__(self, config, message_bus):
        super().__init__("line", config, message_bus)

    async def start(self):
        # 初始化 Line SDK, 注册 Webhook
        pass

    async def stop(self):
        # 清理资源
        pass

    async def send(self, chat_id, content, media=None):
        # 调用 Line Messaging API 发送消息
        pass

    def _convert_to_inbound(self, event):
        # 将 Line Event 转换为 InboundMessage
        pass

```

2. 在 config.json 中添加配置

```

# {
#   "channels": {
#     "line": {
#       "channel_access_token": "xxx",
#       "channel_secret": "xxx"
#     }
#   }
# }

```

3. 注册到 ChannelManager

面试要点：这种“面向接口编程”的适配器模式是设计模式的经典应用。添加新平台不需要修改 Agent 核心代码，符合开闭原则（对扩展开放，对修改关闭）。

9.5 各平台接入指南

9.5.1 Telegram 接入

Step 1: 创建 Bot

1. 在 Telegram 中搜索 @BotFather
2. 发送 /newbot
3. 按提示设置 Bot 名称和用户名
4. 获取 Bot Token (格式: 123456:ABC-DEF...)

Step 2: 配置 config.json

```
{
  "channels": {
    "telegram": {
      "bot_token": "123456:ABC-DEF1234ghIkl-zyx57W2v1u123ew11"
    }
  }
}
```

Step 3: 启动

```
nanobot
# Agent 会自动启动 Telegram Bot 轮询
# 在 Telegram 中找到你的 Bot, 开始对话
```

9.5.2 飞书 (Feishu/Lark) 接入

Step 1: 创建飞书应用

1. 登录 飞书开放平台 (open.feishu.cn)
2. 创建企业自建应用
3. 添加"机器人"能力
4. 获取 App ID 和 App Secret
5. 配置事件订阅 (接收消息事件)
6. 配置消息卡片请求网址

Step 2: 配置 config.json

```
{
  "channels": {
    "feishu": {
      "app_id": "cli_xxxxxxxx",
      "app_secret": "xxxxxxxxxxxxxxxxxxxxxxxx",
      "verification_token": "xxxxxxx",
      "encrypt_key": "xxxxxxx"
    }
  }
}
```

Step 3: 配置 Webhook

飞书需要一个公网可访问的 Webhook URL:
`https://your-domain.com/webhook/feishu`

如果在本地开发, 可以使用 ngrok:
`ngrok http 8080`

9.5.3 钉钉 (DingTalk) 接入

Step 1: 创建钉钉机器人

1. 登录 钉钉开放平台 (open.dingtalk.com)
2. 创建应用 → 添加机器人能力
3. 配置消息接收地址
4. 获取 Robot Code 和 App Key/Secret

Step 2: 配置 config.json

```
{
  "channels": {
    "dingtalk": {
      "app_key": "dingxxxxxxxx",
      "app_secret": "xxxxxxxxxxxxxxxxxxxxxxxx",
      "robot_code": "xxxxxxx"
    }
  }
}
```

9.5.4 Discord 接入

Step 1: 创建 Discord Bot

1. 登录 Discord Developer Portal (discord.com/developers)
2. 创建 Application → Bot
3. 获取 Bot Token
4. 生成邀请链接, 将 Bot 添加到服务器
5. 开启 Message Content Intent

Step 2: 配置 config.json

```
{
  "channels": {
    "discord": {
      "bot_token": "MTIzNDU2Nzg5MDEyMzQ1Njc4OQ.AbCdEf.xxxxxxxxxxxxxxxxxx"
    }
  }
}
```

9.5.5 其他平台

平台	接入难度	关键配置
Slack	中等	Bot Token, Signing Secret
微信	较难	需要企业微信或第三方框架
QQ	较难	QQ 开放平台或第三方框架
Matrix	中等	Homeserver URL, Access Token
Email	简单	IMAP/SMTP 服务器和凭据

9.5.6 多平台同时接入

config.json 中可以同时配置多个平台:

```
{
  "channels": {
    "telegram": {
      "bot_token": "123456:ABC..."
    },
    "discord": {
      "bot_token": "MTIz..."
    },
    "feishu": {

```

```

        "app_id": "cli_xxx",
        "app_secret": "xxx"
    }
}
}

```

启动后，Agent 会同时监听所有配置的平台，通过 session_key 隔离不同平台/用户的会话。

9.6 session_key 会话隔离机制

9.6.1 session_key 的构成

session_key 用于唯一标识一个“对话上下文”：

```
session_key = f"{channel}:{chat_id}"
```

示例：

场景	session_key	说明
CLI 默认	cli:default	命令行交互默认会话
Telegram 私聊	telegram:123456	用户 ID 为 123456
Telegram 群组	telegram:-100789	群组 ID
Discord 频道	discord:guild_1:chan_2	服务器 1 的频道 2
飞书私聊	feishu:ou_xxx	飞书用户

9.6.2 session_key 的作用

session_key 决定了：

1. 会话历史隔离
session_key → sessions/<session_key>.jsonl
不同 key 的用户拥有独立的对话历史
2. 记忆隔离
每个 session_key 独立的短期记忆
但 MEMORY.md 是全局共享的
3. 流式输出路由
Agent 生成的回复按 session_key 路由到正确的平台/会话

9.6.3 session_key_override：线程/子会话隔离

某些平台支持“线程”或“回复链”功能（如 Discord 线程、Slack 线程）。Nanobot 通过 session_key_override 实现线程级别的会话隔离：

```

def build_session_key(message: InboundMessage) -> str:
    """ 构建 session_key，支持线程覆盖 """
    base_key = f"{message.channel}:{message.chat_id}"

    # 如果消息包含线程信息，使用线程级隔离
    thread_id = message.metadata.get("thread_id")
    if thread_id:
        return f"{base_key}:thread_{thread_id}"

    # 如果有显式的 session_key_override
    override = message.metadata.get("session_key_override")
    if override:

```

```
return override
```

```
return base_key
```

会话隔离层次:

```
└─ telegram:123456          # Telegram 用户 123456 的主会话
|
└─ discord:guild_1:chan_2   # Discord 频道的主会话
|   └─ discord:guild_1:chan_2:thread_100 # 线程 100
|       └─ discord:guild_1:chan_2:thread_200 # 线程 200
|           (线程内的对话上下文独立于主频道)
|
└─ feishu:ou_xxx           # 飞书用户的会话
```

面试要点: session_key 的设计体现了“多租户隔离”的思想。每个 session_key 对应独立的会话状态,就像 SaaS 系统中每个租户有独立的数据。线程级覆盖是进一步的细粒度隔离。

9.7 多平台消息流完整链路

9.7.1 完整消息处理流程

以 Telegram 用户发送消息为例:

Step 1: 用户在 Telegram 发送 "帮我查天气"

```
|
[]
```

Step 2: Telegram API 推送 Update 到 Bot

```
|
[]
```

Step 3: TelegramChannel.on_message()

```
└─ 解析 Telegram Update 格式
└─ 构建 InboundMessage:
|   channel: "telegram"
|   sender_id: "123456"
|   chat_id: "123456"
|   content: "帮我查天气"
|   session_key: "telegram:123456"
└─ 发布到 MessageBus Inbound Queue
|
[]
```

Step 4: Agent 从 Inbound Queue 消费消息

```
└─ 加载 session "telegram:123456" 的历史
└─ 加载 MEMORY.md
└─ 构建完整 Prompt
└─ 调用 LLM API
|
[]
```

Step 5: LLM 返回工具调用 → Agent 执行

```
└─ 调用 web_search("北京天气")
└─ 获取搜索结果
└─ LLM 基于结果生成回复
|
[]
```

Step 6: Agent 创建 OutboundMessage

```
└─ channel: "telegram"
└─ chat_id: "123456"
```

```

└─ content: "北京今天晴, 气温 25°C..."
└─ 发布到 MessageBus Outbound Queue
    |
    []
Step 7: ChannelManager 从 Outbound Queue 消费
└─ 路由到 TelegramChannel
└─ _stream_delta 合并 (如果是流式输出)
└─ 调用 TelegramChannel.send()
    |
    []
Step 8: TelegramChannel.send()
└─ 调用 Telegram Bot API
└─ 用户在 Telegram 看到回复

```

9.7.2 跨平台场景

同一个 Agent 可以同时处理来自不同平台的消息:

时间线:

```

t=0s Telegram用户A: "你好"
t=1s Discord用户B: "帮我写代码"
t=2s Agent 处理 A 的消息
t=3s Agent 处理 B 的消息
t=4s 回复发送到 Telegram (给A)
t=5s 回复发送到 Discord (给B)

```

关键: A 和 B 的会话完全独立

session_key_A = "telegram:user_A"

session_key_B = "discord:guild_1:user_B"

9.8 实战练习

练习 1: 接入 Telegram

1. 获取 Bot Token (通过 @BotFather)

2. 配置

```

cat > config.json << 'EOF'
{
  "agents": {
    "defaults": {
      "model": "gpt-4o",
      "provider": "openai"
    }
  },
  "providers": {
    "openai": {
      "api_key": "sk-your-key"
    }
  },
  "channels": {
    "telegram": {
      "bot_token": "your-bot-token"
    }
  }
}

```

EOF

3. 启动

nanobot

4. 在 Telegram 中与 Bot 对话

练习 2: 多平台同时接入

配置同时接入 Telegram 和 Discord

```
cat > config.json << 'EOF'
{
  "agents": {
    "defaults": {
      "model": "gpt-4o",
      "provider": "openai"
    }
  },
  "providers": {
    "openai": {
      "api_key": "sk-your-key"
    }
  },
  "channels": {
    "telegram": {
      "bot_token": "telegram-bot-token"
    },
    "discord": {
      "bot_token": "discord-bot-token"
    }
  }
}
EOF
```

启动后两个平台同时可用

nanobot

练习 3: 观察 session 隔离

启动多平台 Agent 后

在 Telegram 中对话几轮

在 Discord 中对话几轮

然后检查 sessions 目录

```
ls sessions/
```

```
# telegram:123456.jsonl
```

```
# discord:guild_1:chan_2.jsonl
```

验证两个会话的历史是独立的

```
wc -l sessions/*.jsonl
```

9.9 面试高频题

题目 1: 如何设计一个支持多平台的 Agent 系统?

参考回答:

“我会采用 Nanobot 的架构思路，核心是三层解耦设计：

第一层是 **Agent** 核心，只处理业务逻辑——接收统一格式的消息、调用 LLM、执行工具、生成回复。它完全不知道消息来自哪个平台。

第二层是 **MessageBus** 消息总线，采用双队列架构——Inbound Queue 收集所有平台的入站消息，Outbound Queue 收集 Agent 的出站回复。队列实现生产者/消费者解耦，天然支持异步和背压。

第三层是 **Channel** 适配器，每个平台一个 Channel 实现，继承统一的 BaseChannel 抽象类。Channel 负责两件事：一是将平台特定的消息格式转换为统一的 InboundMessage，二是将 OutboundMessage 转换为平台特定的 API 调用。

添加新平台只需实现一个新的 Channel 类，不需要修改 Agent 和 MessageBus 代码，符合开闭原则。

会话隔离通过 session_key（格式 {channel}:{chat_id}）实现，每个 session_key 对应独立的对话历史和状态。

出站优化方面，ChannelManager 负责流式输出的降频合并（避免平台 API 限流）和发送失败的指数退避重试。”

题目 2: MessageBus 和直接调用有什么区别？

参考回答：

“直接调用意味着 Agent 直接引用并调用 Channel 的发送方法，这会导致紧耦合——Agent 需要知道所有平台的存在。每添加一个平台都要修改 Agent 代码。

MessageBus 通过异步队列解耦了生产者和消费者。Channel 往 Inbound Queue 里放消息，Agent 从 Inbound Queue 取消息；Agent 往 Outbound Queue 里放回复，ChannelManager 从 Outbound Queue 取出后路由到对应 Channel。双方通过数据结构（InboundMessage/OutboundMessage）交互，互不依赖。

这种设计还带来了背压控制——如果 Agent 处理不过来，消息会在队列里排队，不会丢失。在测试时也更方便，可以直接向队列注入消息而不需要启动真实的平台连接。”

题目 3: 如何处理不同平台的消息格式差异？

参考回答：

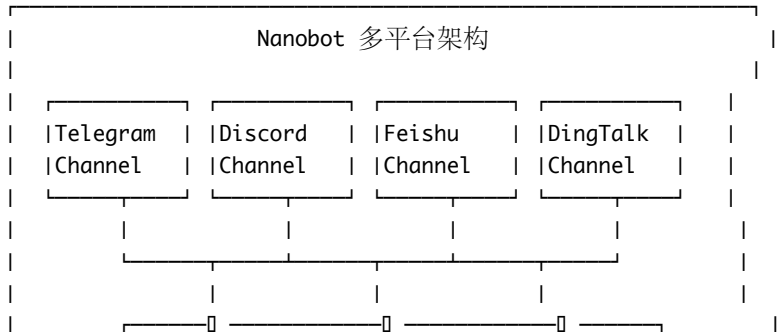
“通过适配器模式。定义一个统一的消息结构——InboundMessage 包含 channel、sender_id、chat_id、content、media、metadata 等字段。每个平台的 Channel 实现一个 _convert_to_inbound 方法，负责将平台特定格式（如 Telegram 的 Update、Discord 的 Message Event）映射到统一格式。

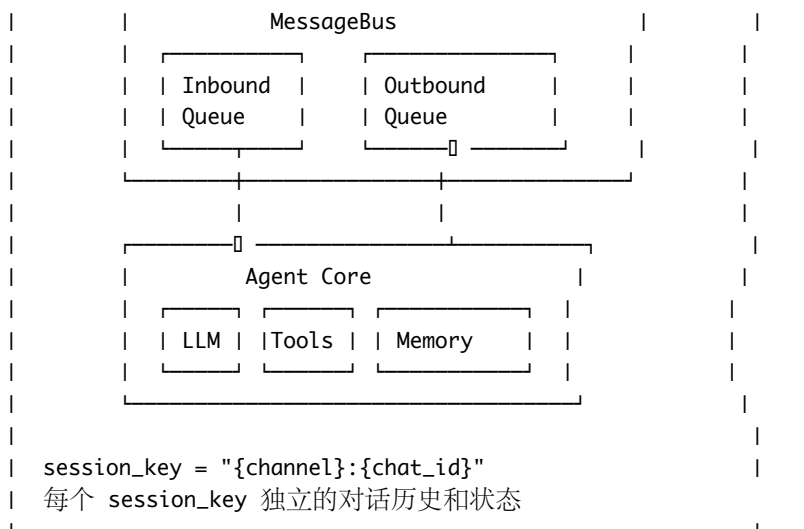
出站同理，OutboundMessage 是统一格式，Channel 的 send 方法负责转换为平台 API 调用。

对于平台特有的功能（如 Telegram 的 Inline Keyboard、Discord 的 Embed），可以放在 metadata 字段中传递，Channel 在发送时识别并使用。”

9.10 本章小结

核心架构图





面试记忆清单

考点	一句话回答
架构设计	三层解耦：Agent 核心 + MessageBus + Channel 适配器
MessageBus	异步双队列（Inbound + Outbound），解耦生产者/消费者
InboundMessage	统一入站消息：channel, sender_id, chat_id, content, media, metadata
OutboundMessage	统一出站消息：channel, chat_id, content, reply_to, media
BaseChannel	抽象基类，每个平台实现一个适配器（适配器模式）
session_key	<code>{channel}:{chat_id}</code> ，实现会话隔离
流式优化	<code>_stream_delta</code> 降频合并，间隔 300ms
失败重试	指数退避：1s, 2s, 4s, 8s, 16s
添加新平台	只需实现 BaseChannel 子类，开闭原则

下一章：10 - 子 Agent 与定时任务 —— 深入理解 SubAgent 后台任务和 Cron 定时调度